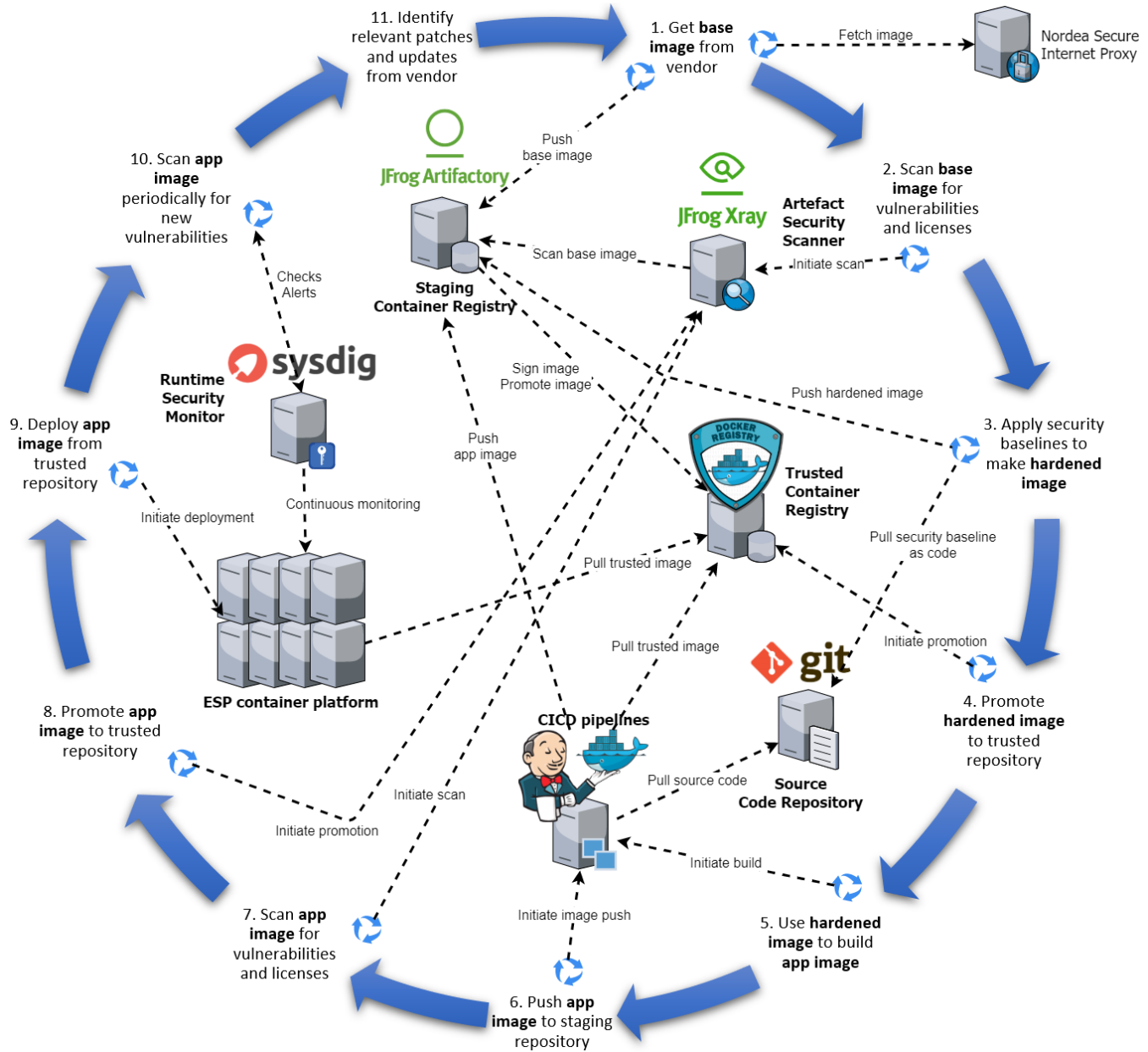


Container image strategy



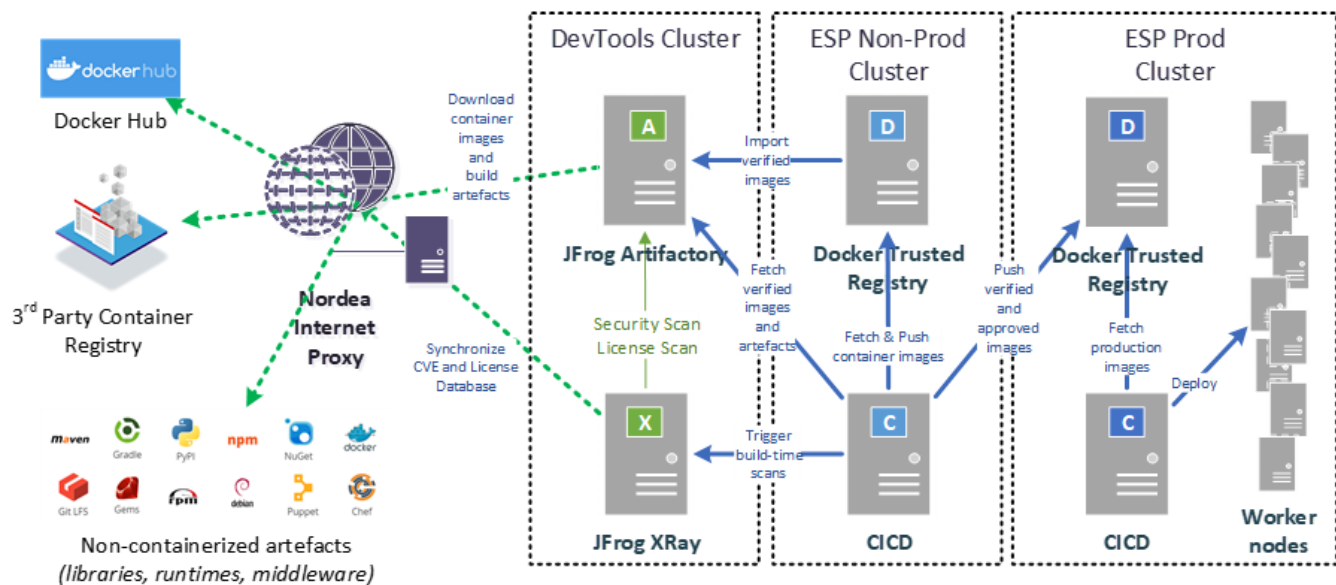
1. Introduction

The lifecycle of container images in ESP is built around capabilities of solutions that represent container repositories (e.g. Docker registry). In ESP there are two repository solutions - JFrog Artifactory (hereafter - ART) and Docker Trusted Registry (hereafter - DTR). The difference between them in terms of use is next:

1. DTR is managed by ESP team and used as trusted registry, which holds base container images that passed verification and validation.
2. ART is managed by CID team and used as staging registry, which holds non-verified and intermediate container images derived either from 3rd party or built by developers.
3. ART also used as repository for non-container artefacts - Java (Maven / Gradle), .NET / NuGet, Python (PyPi), Javascript (NPM), GO, Debian, RPM, and so on.

It is possible that at some point DTR will be decommissioned in favour to ESP-managed trusted repositories in ART. However, as of today in Q1 2021 the DTR is tightly coupled with existing ESP setup, which originates from initial setup of the Docker Enterprise platform. Docker Enterprise or Docker EE platform has recently been renamed to Mirantis Kubernetes Engine (hereafter - MKE).

2. Sourcing and acquisition



The preferred way to acquire container images is to use Remote Repository feature in JFrog Artifactory. ART uses Nordea Security Proxy for indirect access to 3rd party container registries, such as DockerHub or Red Hat UBI repositories. So, ART can proxy a remote Docker registry through remote repositories. A Remote Repository defined in ART serves as a caching proxy for a registry managed at a remote URL such as <https://registry-1.docker.io/> or other custom Docker registry. Container images requested from a remote repository are cached on demand but it is one-way download only. It is not possible to manually push container images back to a remote container repository.

Base images obtained and cached in ART Remote Repository cannot be used directly to build applications. In order to use them as base those images must be:

1. Scanned for security vulnerabilities (CVEs hereafter)
2. Scanned for license compliance
3. Hardened by applying Nordea security baselines

As result of above steps, the original image should appear in DTR under different image tag than original. However, a scheme for the new tag after hardening does incorporate the original tag to avoid confusion.

In order to get new remote repository added to ART the interested party should contact ESP team and to agree on conversion and tag naming scheme for hardened images.

Other ways to acquire and ingest base images, such as one-time downloading from Internet or manual uploading of the image from developer laptop to ART must be discussed and agreed with ESP and CI/CD teams.

3. External container repositories

Docker Certified Official Images from **Verified Publishers** and **Red Hat Certified Universal Base Images** are **primary** choices for selecting 3rd party base container images for all ESP apps. In order to ensure that the container image originates from the vendor and not from unknown or malicious source Docker Hub introduces several levels of trust and certification.



1. **Docker Certified** logo – assures that the qualified content is conforming to best practices and pass certain baseline tests for container images. The Docker Certification program is designed for enterprise customers to recognize high-quality content, which is compatible, tested, and supported on MKE by a Verified publisher. Docker Certification is aligned to the available MKE, and gives enterprises a trusted way to run more technology in containers with the confidence knowing there is a collaborative support relationship in place between Docker and the Verified Publisher



2. **Verified Publisher** logo – assures that publisher of the container images is an official partner of the vendor of MKE product – Mirantis Inc. Verified Publishers have engaged directly with Docker's partner team and enrolled in our Docker Technology Partner program. These profiles indicate that the business entity of the publisher has been validated and have taken proactive steps to partner with Docker. Unlike Community content, these Verified Publisher profiles indicate true provenance of the content.



3.

Official Image logo – assures that the container image originates from verified publisher, created using best practices, tested and properly documented. The Docker Official Images are a curated set of Docker repositories hosted on Docker Hub.

Each of the images in the Official Images is scanned for vulnerabilities on Docker Hub. The results of these security scans provide valuable information about which images contain security vulnerabilities, and allow to choose images that align with security standards.

Pretty much the same model applies to Red Hat Universal Base Images (UBI). UBIs are OCI-compliant (Docker-compatible) container base operating system images with complementary runtime languages and packages that are freely redistributable. UBIs are built from portions of Red Hat Enterprise Linux. UBI images can be obtained from the Red Hat container catalogue.

In order to meet enterprise customers' security and support requirements, software vendors take advantage of **Red Hat Container Certification**. Once certified, partner applications are fully supported by Red Hat when deployed on container platforms.

Non-certified 3rd party container images still can be used by developers if necessary. However, in order to be used in Nordea environment they must pass mandatory security and license compliancy scanning and meet Nordea security standards. All 3rd party container images downloaded from Internet must be routed through **Nordea Secure Internet Proxy** that implements security scanning capabilities (e.g. malware scan).

If the 3rd party base image is subject for commercial licensing (e.g. some of products from Oracle, IBM, etc) the distribution channel for such vendor-supplied image is negotiated in the commercial agreement. ESP and CIDC teams help project teams to acquire that kind of container image and other vendor-supplied artefacts using agreed distribution channels and repositories.

4. Ingestion-time scanning

The CVE and license scanning is performed by JFrog XRay solution (hereafter - XRAY). XRAY is a universal software composition analysis (SCA) solution that natively integrates with ART, giving developers and DevSecOps teams an easy way to proactively identify CVEs on open source and license compliance violations, before they come to surface in production releases. Since ART is not only container image repository the same CVE scanning also applied to non-container artefacts - JAR/WAR, NuGet, NPM, RPM, etc.

XRAY keeps its license and CVE databases in sync with masters therefore ingestion-time scanning of the container images is performed against as latest database as possible. Each CVE identified by XRAY is associated with severity rank - Minor, Major, Critical. As the rule, 3rd party container images with CVEs of Critical severity are rejected.

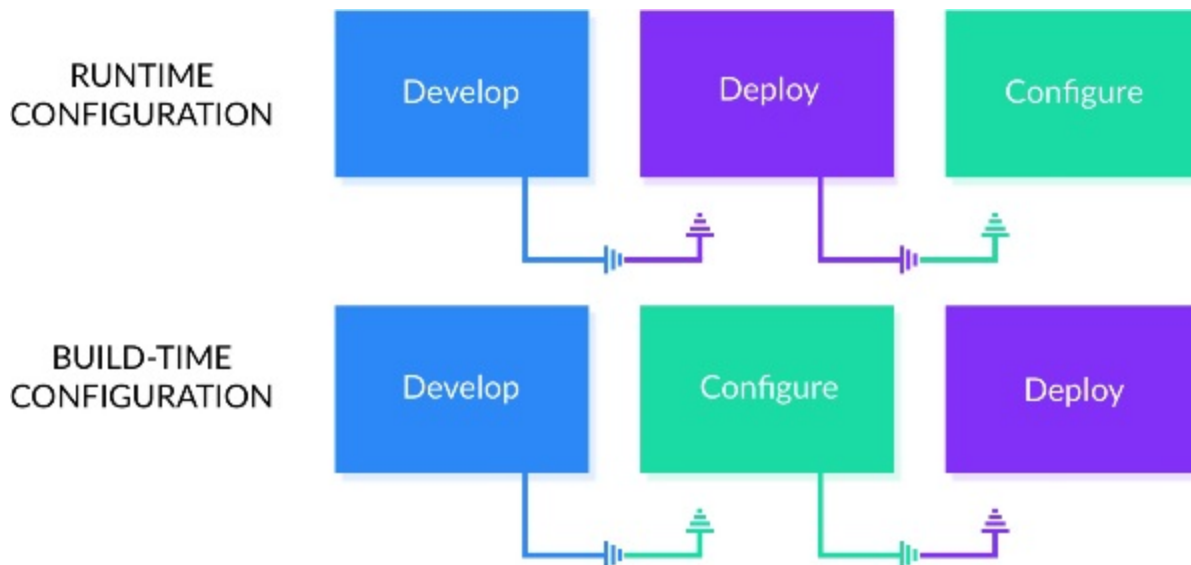
The license compliance check is performed by XRAY through identifying the set of licenses accompanying 3rd party or open source component in container image. Each identified license type is matched against set of rules that may restrict the usage of the certain licenses in Nordea solutions. At the moment the XRAY license check is non-restrictive informative procedure.

5. Base image hardening

The hardening process for base container images is addressing the following topics:

1. Adoption of the 3rd party container images to Nordea infrastructure setup - from installing Nordea CA certificates to configuring Nordea-specific services and endpoints.
2. Enhancements to the default configuration of applications, middleware and engines embedded into 3rd party containers. Instructions on how the default configuration should be changed are called **security baselines** and provided by Nordea Cyber Security Platforms unit.

In result of the hardening process the new base image is produced, digitally signed and promoted to DTR. New image is a direct derivative from the 3rd party base image and incorporates original image tag within the naming scheme (e.g. *ubuntu:xenial-20210114* *nordea:ubuntu-xenial-20210114* or similar). For more details regarding tag naming conventions after hardening please refer to ESP documentation.



The hardening process is designated for build-time configuration. Baseline configuration defaults can be modified in subsequent builds of developer images and/or changed in run-time too. So, ESP provides a set of additional means to ensure that the eventual configuration (build-time + runtime) still conforms to the baseline and mandatory security requirements even after subsequent modifications of the initial hardened configs by developers. These tools are described later in "Run-time scanning" section.

In order to affirm that container image was scanned and hardened the container signatures are used. Container signatures serve as certification token and are verified during build and deployment phases to ensure that only approved/trusted base images are used. Container signatures are different for production and non-production environment because CVE severity and other thresholds might be different for development/testing and preprod/production environments. For example, it might be possible to use base images with Major CVEs in Dev but not in Prod, etc.

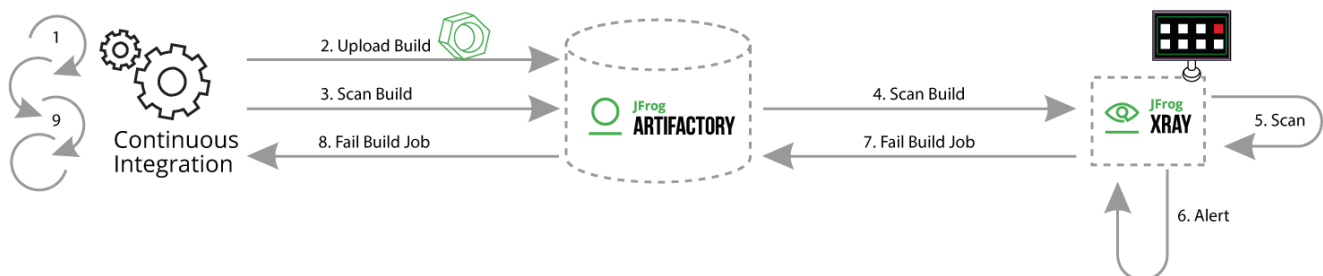
The application of container signatures and management of the PKI associated with them is done via Docker Content Trust (hereafter - DCT) solution, which is an integrated part of DTR. That's another reason why DTR is used as trusted registry to keep approved base images. The X509 certificate management associated with DTR signature mechanism is done by Nordea PKI as specified by X500 standard, including enrolment, verification and revocation of certificates and associated digital signatures.

6. Build-time scanning

Once the 3rd party base container image is ingested, scanned, hardened, signed, and promoted to trusted registry it can be used by developers for building applications. Developer applications are produced from the combination of Nordea-developed source code and 3rd party libraries, which usually are specific to selected programming language (e.g. JAR / WAR files, NuGet, PyPi, NPM, etc). The responsibility of the application build process belongs to CICD subsystem that is based on Jenkins automation server.

3rd party libraries and artefacts used in conjunction with source code for application builds are stored in ART and undergo the same XRAY security and license scanning procedures. However, the lifecycle of language-specific artefacts and OS packages (e.g. .DEB / RPM) is not in scope of this strategy, which is focused on container images.

When application is built by CICD the next phase is to create the container image for it. Container image should include all components required for its standalone run as self-sufficient deployment unit. It means that CICD-produced application package or artefact is combined with chosen base image and other packages that might be installed and configured in container build time through assembly script called Dockerfile. Dockerfile is a text document that contains all the commands a user could call on the command line to assemble an image.

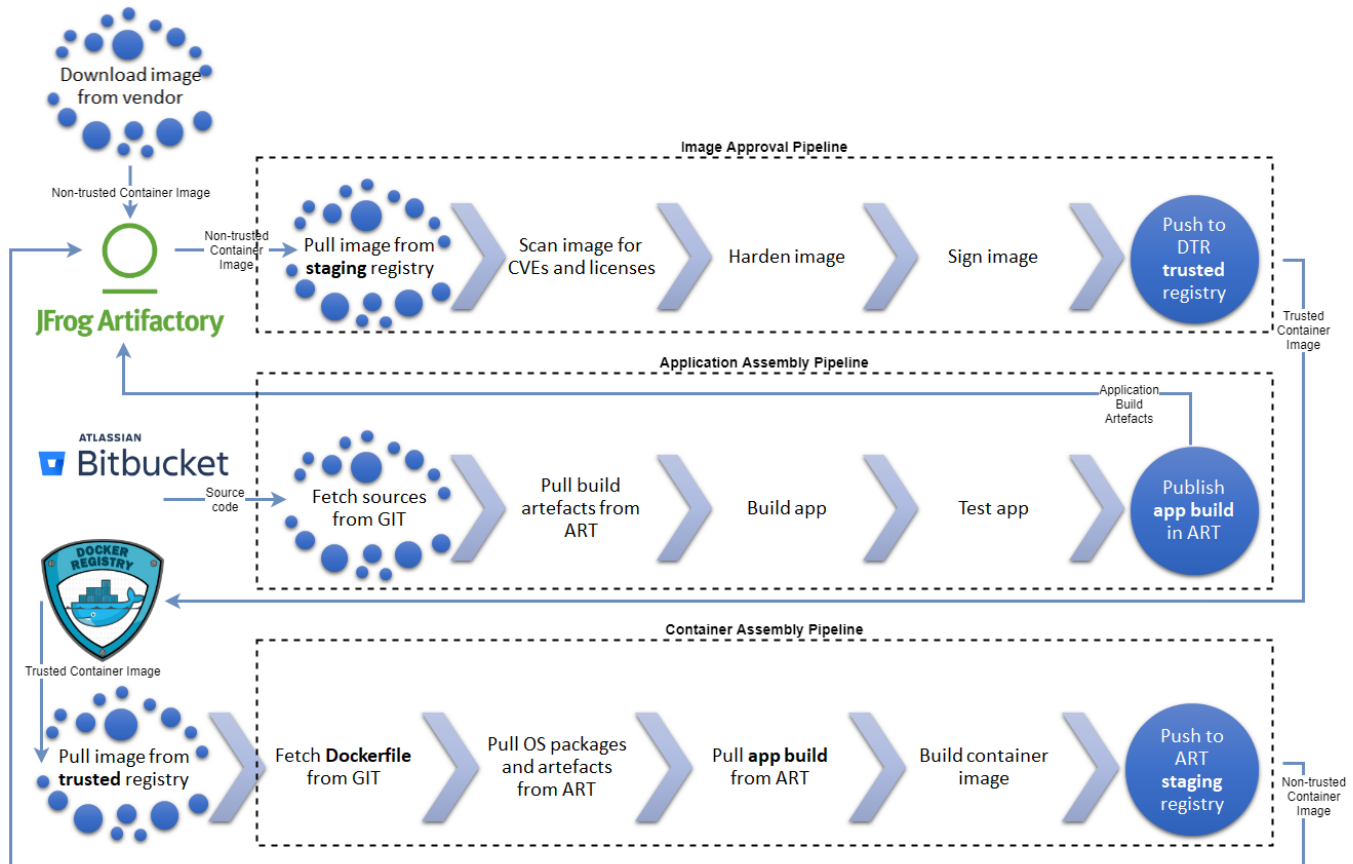


Result of the app container creation phase is a new container image that incorporates base image and developer-written application with all the required libraries and other components. New container image is uploaded to staging registry in ART and triggers a new round of security and license scanning to obtain validation signature and promote the image to the trusted in DTR.

Deployments of Nordea-developed container images are originated only from DTR, where new container image can land only after necessary scans and verifications.

7. Nordea-developed base images

Base container images can be originated from 3rd party vendor or from Nordea developers. Usually internally-developed base images created to facilitate multi-stage development, so that multiple applications of similar kind can share the same base. For example, Core Banking Platform (hereafter - CBP) apps are using 3rd party Temenos Transact engine (hereafter - T24). In order to facilitate creation of CBP apps the developers might first create a shared T24 image for all of derived business applications, so that there is no need to adopt 3rd party vendor image each time when new CBP-based app is introduced.



In fact, there is no big difference between downloading base image from 3rd party or pushing Nordea-developed base image to staging repository. Both intents trigger the same process of scanning and hardening, which is indicated as "Image Approval Pipeline" on the diagram above.

The core difference between base images made by Nordea developers is whether the produced image is supposed to be consumed within a single project scope (i.e. apps included in one solution) or it is declared as system-wide base image and can be reused between multiple projects across Nordea.

The rule of thumb is – the team created a new base image is responsible for the maintenance of its further lifecycle, including bugfixes, regular patches /upgrades and decommissioning when it is no longer used.

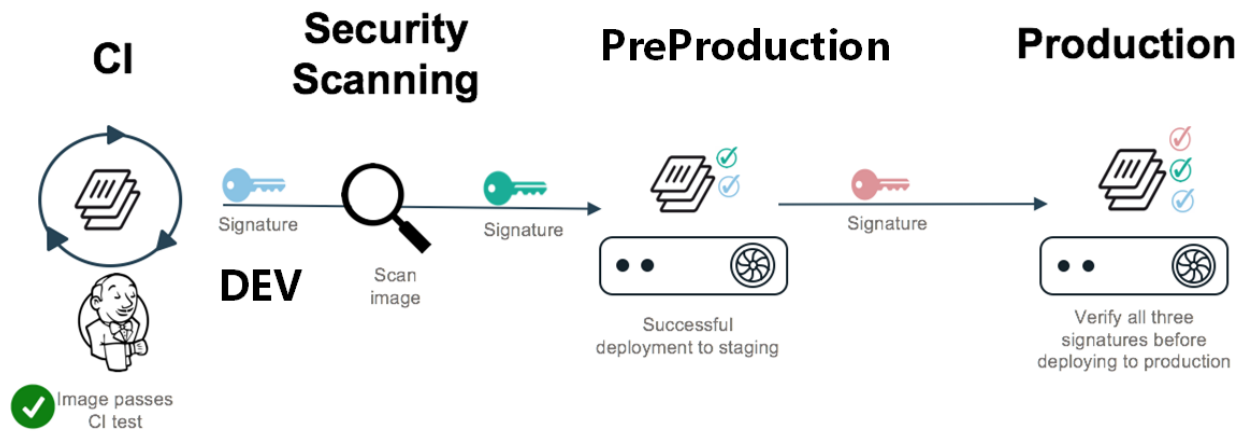
8. Deployment-time scanning

Deployment time scanning consists from two phases:

1. Validation of the container signature
2. Security scanning of the container image against latest CVE databases (**production-only**)

So, container deployments do perform signature verification of container images to ensure that they passed all required post-build checks and haven't been modified at rest after promotion to DTR. In order to minimize risks of leaking newly discovered CVEs which were unknown at the build time the production deployments also perform another round of CVE scanning as prerequisite for production readiness.

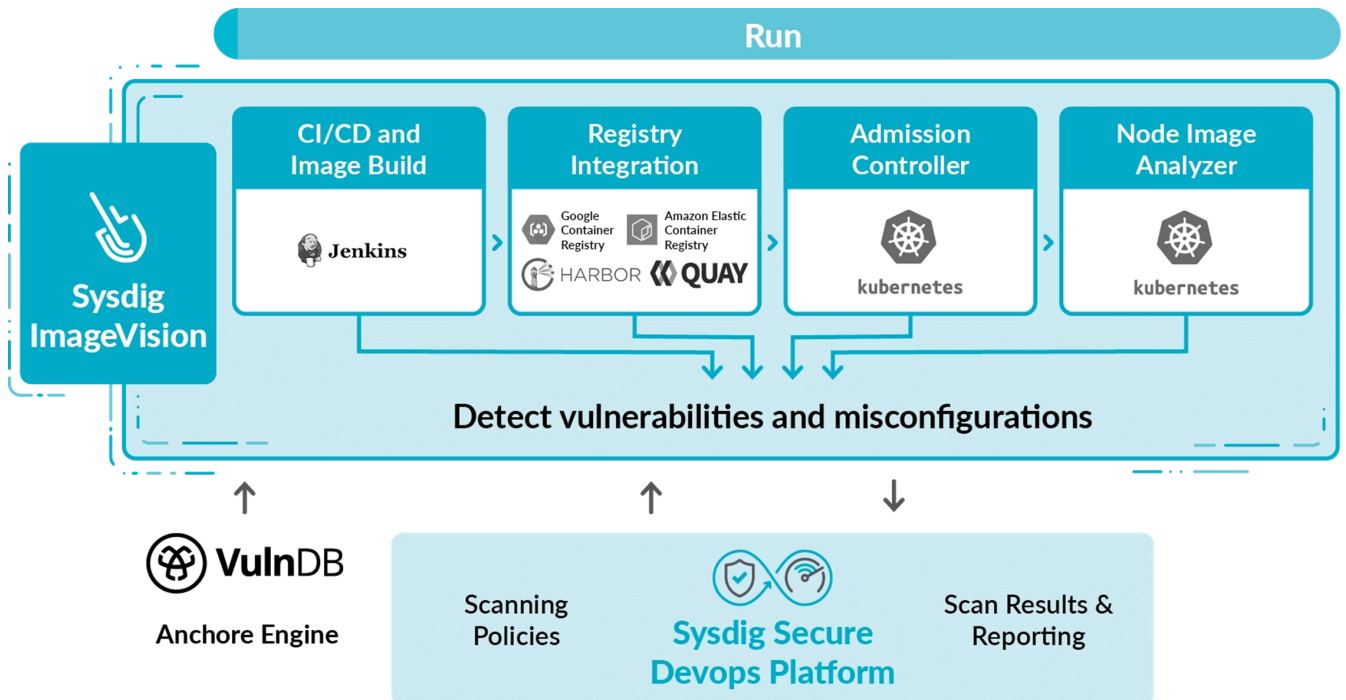
As the rule, production deployments fail if the application container image contains one or more critical-ranked CVE discovered between container DTR promotion phase and deployment request.



Note that different deployment environments – dev, pre-prod, and prod – have different signatures because the safety and security thresholds between those envs are different. Ultimately each container image deployed to production shall collect all lower-grade signatures for risk minimization purposes.

9. Run-time scanning

Run-time scanning and monitoring is performed by secure container monitoring tools called Sysdig Secure (hereafter SDS).



There are three aspects of runtime security monitoring:

1. Periodical vulnerability scans performed each time when CVE databases are renewed. Those are needed to ensure that running applications are up to date and do not contain fresh CVE discovered after deployment.
2. Configuration scans performed on each change of the deployment configuration parameters to ensure post-deployment conformance of configuration to the build-time security baselines.
3. Compliance to SDS policies defined as set of Falco rules in order to avoid running of potentially vulnerable, data leaking or misconfigured containers.

Every mismatch in steps specified above automatically triggers an incident within SDS framework. Each incident initiates a predefined set of automated actions – from alerting and informing support lines to automatic pause, restart or killing the running container to avoid potential data breach.

SDS policies is an additional level of security that performs multi-layer checks of container images, their deployment configurations and run-time environment setup to identify potentially vulnerable, non-compliant or unwanted combinations. That is; an SDS policy is a combination of rules about

activities Nordea wants to detect in a deployment environment, the actions that should be taken if the policy rule is breached, and-- potentially-- the notifications that should be sent.

Example of Falco rule:

```
- rule: Detect shells in a container

desc: Now allowed to run a shell in a production container

condition: container.id != host and proc.name in (system_shells)

output: Bash ran inside a container (user=%user.name command=%proc.cmdline %container.info)

priority: INFO
```

The expression language that SDS uses for Falco rules is YAML. Security policies are defined by CISO and Cyber Security team and stored as YAML code in source code repository (Git / Bitbucket) in order to efficiently use them in CICD/GitOps lifecycle. So, they undergo the same processes as any other source code – commits, versioning, pull requests and so on.

10. Alerting and automated incident resolution

Alerting and incident resolution in ESP is pretty much based on capabilities of SDS to identify in runtime the various incompliances in the containers, in their deployment configuration, and malicious behavioural patterns, which may indicate security breaches such as hacker intrusion or malware/virus outbreak.

SDS defines the following set of alerting and incident resolution actions that can be performed in response to triggering from Falco.

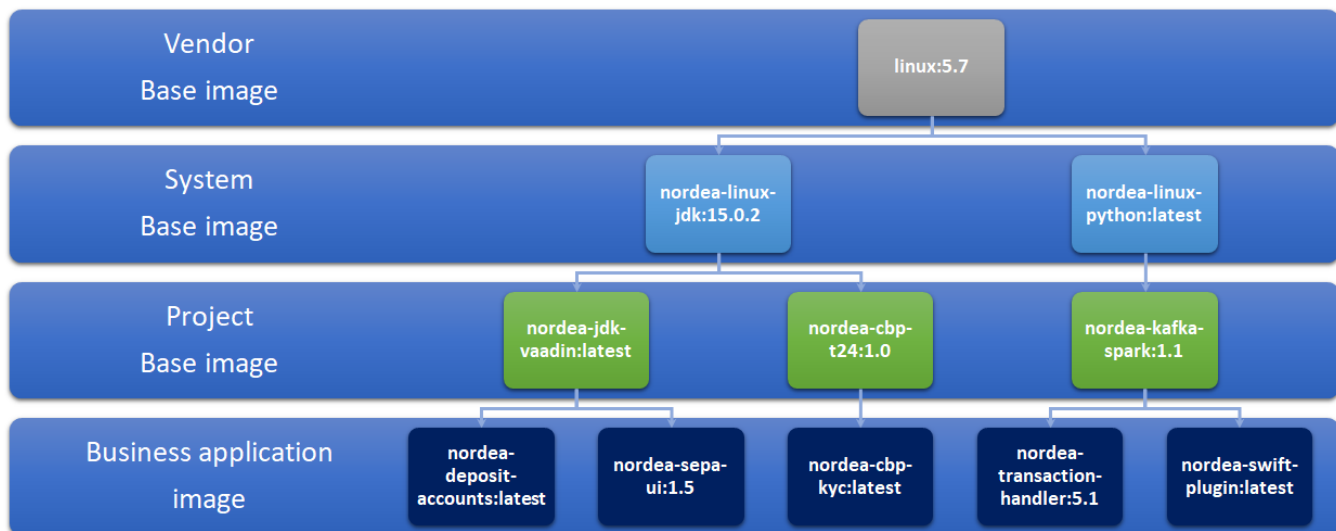
1. **Containers.** What should happen to affected containers if the policy rules are breached:
 - **Nothing** (alert only): Do not change the container behaviour; send a notification according to Notification Channel settings.
 - **Kill**: Kills one or more running containers immediately.
 - **Stop**: Allows a graceful shutdown (10-seconds) before killing the container.
 - **Pause**: Suspends all processes in the specified containers.
2. **Capture**: Enable activity captures in case of an event and define the number of seconds before and after the event that should be in the snapshot.
3. **Notification**: Chose one or more a notification channels for sending notification of events to appropriate personnel.

All actions above can be combined into the cascade sets of responses. Whenever automated action is triggered it often may lead to the need to update the application & container images via initiating a new CICD build round.

11. Patching, updating, and dependencies

The rule of thumb is - team that introduces a new base image is responsible for its patching and upgrading.

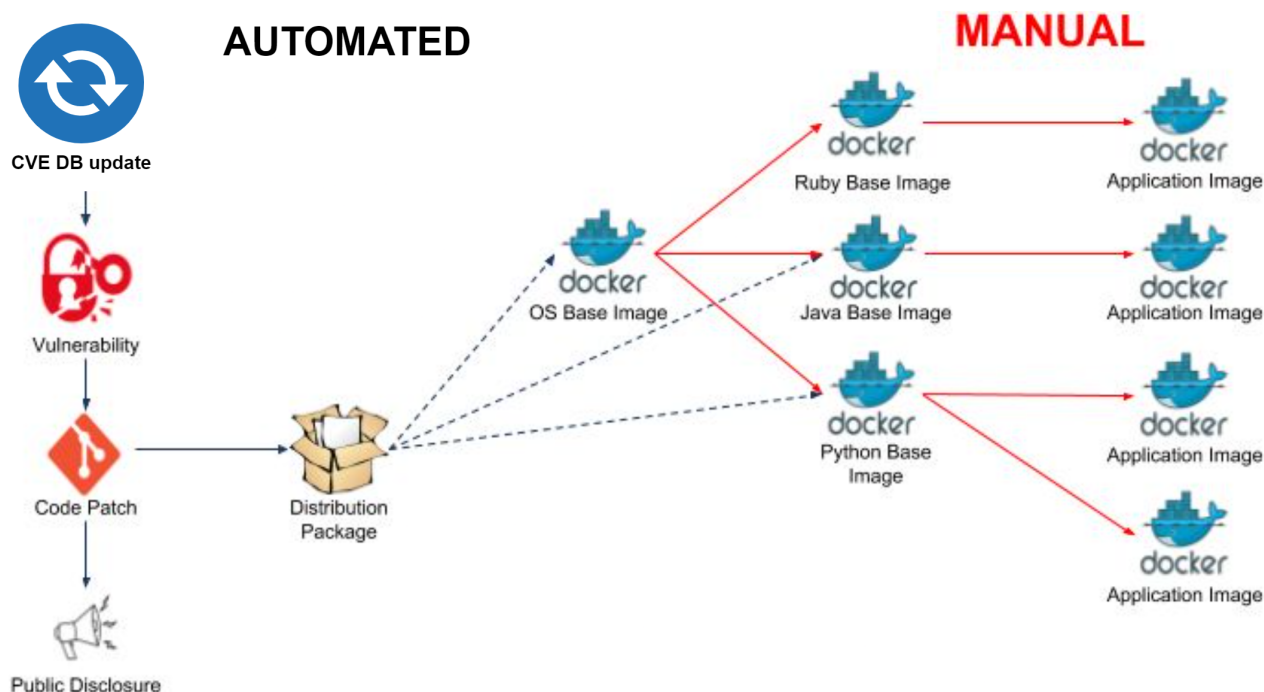
The patching process in overall is rather cascaded and hierarchical than linear. That is; the number of base images that can derive from one vendor image is not limited, so potentially any team can create a distinctive version of vendor-supplied base image by adding their own OS-packages and other components inside.



The hierarchical nature of container image dependencies creates 3 side effects:

1. Inheritance of base images increases reliability when they are frequently patched and updated, in line with detection of new CVEs, bugs and other defects.
2. Inheritance of base images may decrease reliability when they are kept outdated for a long time. It means that old CVEs, bugs and defects will stay in child images as long as the whole dependency tree is not patched.
3. Manageability of hierarchical patching process is high if the cascade patching (from parent to children) is not automated.

The problem of cascade patching is outlined on diagram below:



Even though the CVE scanning and patch application processes are automated there is no similar automated process for hierarchical application of patches from parent to children images at the moment.

Therefore the proposed patching strategy is reverse comparing to build inheritance process.

That is; when security scan detects CVE(s) in built or running container the team who owns that container image takes care of arranging the application of relevant patches to the whole chain from top to the bottom.

1. If CVE detected in the container image is local-scope only and doesn't originate from previous iteration of base image(s) then the patching is performed only to one container image
2. If detected CVE derives from another base container belonging to the same team, the whole chain must be patched.
3. If base container with CVE is coming from another team it is necessary to inform that owner team to apply relevant patches using rules 1. and 2.

The same flow is valid for non-CVE patches, hotfixes, bugfixes and updates (e.g. OS-package upgrades inside the container image).

In this kind of strategy it is important to stick to the single container ownership - one team is responsible for the container image. Otherwise it would be hard to avoid collisions and race conditions if multiple teams will end up patching the same container image.

In any case, artefact-wise there is no collisions at all since every patch produces a new unique container image every time. However, the natural discrepancies in tag naming conventions (if done by different teams for same container images) may create unnecessary duplications and alternatively (mis)configured images. So a single ownership of the container image is highly recommended.

ESP team understands that current manual patching strategy is not perfect and is looking for a better automated way of cascade patching. The biggest issue with fully automated patching is - regression testing that in many cases still needs to be performed manually by teams. So, as of today Q1 2021 the proposed patching strategy for container images is - manual, following dependencies from top to the bottom.